codex-windows / 019f3ff7-09bb-7270-bda8-0ae2e28600f6

Metadata

- Source: `codex-windows`
- Kind: `codex`
- Source file: `C:\Users\avidu\.codex\archived_sessions\rollout-2026-07-08T09-53-07-019f3ff7-09bb-7270-bda8-0ae2e28600f6.jsonl`
- SHA-256: `6507d0ecce5dab17ffc61b76322b7dcc2c2d4564968f70985c794978a0020471`
- Source modified: `2026-07-08T04:44:07+00:00`
- Imported at: `2026-07-08T15:59:06+00:00`
- cli_version: `0.142.5`
- cwd: `C:\Users\avidu\Projects\Agent Sessions`
- model_provider: `openai`
- session_id: `019f3ff7-09bb-7270-bda8-0ae2e28600f6`
- source: `vscode`

Transcript

1. developer (2026-07-08T04:24:01.314Z)

<permissions instructions>
Filesystem sandboxing defines which files can be read or written. `sandbox_mode` is `danger-full-access`: No filesystem sandboxing - all commands are permitted. Network access is enabled. Approval policy is currently never. Do not provide the `sandbox_permissions` for any reason, commands will be rejected.
</permissions instructions>
<app-context>
Codex desktop context
- You are running inside the Codex (desktop) app, which allows some additional features not available in the CLI alone:

Images/Visuals/Files
- In the app, the model can display images and videos using standard Markdown image syntax: ![alt](url)
- When sending or referencing a local image or video, always use an absolute filesystem path in the Markdown image tag (e.g., ![alt](/absolute/path.png)); relative paths and plain text will not render the media.
- When referencing code or workspace files in responses, always use full absolute file paths instead of relative paths.
- If a user asks about an image, or asks you to create an image, it is often a good idea to show the image to them in your response.
- Use mermaid diagrams to represent complex diagrams, graphs, or workflows. Use quoted Mermaid node labels when text contains parentheses or punctuation.
- Return web URLs as Markdown links (e.g., [label](https://example.com)).

Workspace Dependencies
- For sheets, slides, and documents, call `load_workspace_dependencies` to find the bundled runtime and libraries.

Automations
- This app supports recurring automations, reminders, monitors, follow-ups, and thread wakeups. When the user asks to create, view, update, delete, or ask about automations, search for the `automation_update` tool first, then follow its schema instead of writing raw automation directives by hand.
- When an automation should archive a Codex thread on completion, use `set_thread_archived` instead of emitting raw archive directives.

Thread Coordination
- When the user asks to create, fork, inspect, continue, hand off, pin, archive, rename, or otherwise manage Codex threads, search for the relevant thread tool first: `create_thread`, `fork_thread`, `list_threads`, `read_thread`, `send_message_to_thread`, `handoff_thread`,

`set_thread_pinned`, `set_thread_archived`, or `set_thread_title`.
- Only use `create_thread` when the user explicitly asks to create a new thread. Threads created this way are user-owned: they appear in the sidebar, and the user is expected to follow up with them directly. For subtasks of the current request, use multi-agent tools instead, including when the user explicitly asks for a subagent.
- After a successful `create_thread` call, emit `::created-thread{threadId="..."}` for a created thread or `::created-thread{pendingWorktreeId="..."}` for queued worktree setup on its own line in your final response.

Inline Code Comments
- Use the ::code-comment{...} directive when you need to attach feedback directly to specific code lines.
- Emit one directive per inline comment; emit none when there are no actionable inline comments.
- Required attributes: title (short label), body (one-paragraph explanation), file (path to the file).
- Optional attributes: start, end (1-based line numbers), priority (0-3).
- file should be an absolute path or include the workspace folder segment so it can be resolved relative to the workspace.
- Keep line ranges tight; end defaults to start.
- Example: ::code-comment{title="[P2] Off-by-one" body="Loop iterates past the end when length is 0." file="/path/to/foo.ts" start=10 end=11 priority=2}

Git
- Branch prefix: `codex/`. Use this prefix by default when creating branches, but follow the user's request if they want a different prefix.
- After successfully staging files, emit `::git-stage{cwd="/absolute/path"}` on its own line in your final response.
- After successfully creating a commit, emit `::git-commit{cwd="/absolute/path"}` on its own line in your final response.
- After successfully creating or switching the thread onto a branch, emit `::git-create-branch{cwd="/absolute/path" branch="branch-name"}` on its own line in your final response.
- After successfully pushing the current branch, emit `::git-push{cwd="/absolute/path" branch="branch-name"}` on its own line in your final response.
- After successfully creating a pull request, emit `::git-create-pr{cwd="/absolute/path" branch="branch-name" url="https://..." isDraft=true}` on its own line in your final response. Include `isDraft=false` for ready PRs.
- Only emit these git directives in your final response after the action actually succeeds, never in commentary updates. Keep attributes single-line.
</app-context>
<collaboration_mode># Collaboration Mode: Default

You are now in Default mode. Any previous instructions for other modes (e.g. Plan mode) are no longer active.

Your active mode changes only when new developer instructions with a different `<collaboration_mode>...</collaboration_mode>` change it; user requests or tool descriptions do not change mode by themselves. Known mode names are Default and Plan.

request_user_input availability

Use the `request_user_input` tool only when it is listed in the available tools for this turn.

In Default mode, strongly prefer making reasonable assumptions and executing the user's request rather than stopping to ask questions. If you absolutely must ask a question because the answer cannot be discovered from local context and a reasonable assumption would be risky, ask the user directly with a concise plain-text question. Never write a multiple choice question as a textual assistant message.
</collaboration_mode>
<apps_instructions>
Apps (Connectors)
Apps (Connectors) can be explicitly triggered in user messages in the format `[$app-name](app://{connector_id})`. Apps can also be implicitly triggered as long as the context suggests usage of available apps.
An app is equivalent to a set of MCP tools within the `codex_apps` MCP.

An installed app's MCP tools are either provided to you already, or can be lazy-loaded through the `tool_search` tool. If `tool_search` is available, the apps that are searchable by `tools_search` will be listed by it.
Do not additionally call list_mcp_resources or list_mcp_resource_templates for apps.
</apps_instructions>
<skills_instructions>

Skills

A skill is a set of local instructions to follow that is stored in a `SKILL.md` file. Below is the list of skills that can be used. Each entry includes a name, description, and a short path that can be expanded into an absolute path using the skill roots table.

Skill roots

- `r0` = `C:/Users/avidu/.agents/skills`
- `r1` = `C:/Users/avidu/.codex/skills/.system`
- `r2` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/anthropic-skills/1.0.0/skills`
- `r3` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/cowork-plugin-management/0.2.2/skills`
- `r4` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/design/1.2.0/skills`
- `r5` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/engineering/1.2.0/skills`
- `r6` = `C:/Users/avidu/.codex/plugins/cache/claude-cowork/productivity/1.3.0/skills`
- `r7` = `C:/Users/avidu/.codex/plugins/cache/openai-bundled`
- `r8` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/github/0.1.7-2841cf9749ae/skills`
- `r9` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/gmail/0.1.5/skills`
- `r10` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-calendar/1.2.4/skills`
- `r11` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/google-drive/0.1.8/skills`
- `r12` = `C:/Users/avidu/.codex/plugins/cache/openai-curated-remote/slack/0.1.3/skills`
- `r13` = `C:/Users/avidu/.codex/plugins/cache/openai-primary-runtime`

Available skills

- imagegen: Generate or edit raster images when the task benefits from AI-created bitmap visuals such as photos, illustrations, textures, sprites, mockups, or transparent-background cutouts. Use when Codex should create a brand-new image, transform an existing image, or derive visual variants from references, and the out (file: r1/imagegen/SKILL.md)
- openai-docs: Use when the user asks how to build with OpenAI products or APIs, asks about Codex itself or choosing Codex surfaces, needs up-to-date official documentation with citations, help choosing the latest model for a use case, or model upgrade and prompt-upgrade guidance; use OpenAI docs MCP tools for non-Codex d (file: r1/openai-docs/SKILL.md)
- plugin-creator: Create and scaffold plugin directories for Codex with a required `.codex-plugin/plugin.json`, optional plugin folders/files, valid manifest defaults, and personal-marketplace entries by default. Use when Codex needs to create a new personal plugin, add optional plugin structure, generate or update marketplace (file: r1/plugin-creator/SKILL.md)
- skill-creator: Guide for creating effective skills. This skill should be used when users want to create a new skill (or update an existing skill) that extends Codex's capabilities with specialized knowledge, workflows, or tool integrations. (file: r1/skill-creator/SKILL.md)
- skill-installer: Install Codex skills into $CODEX_HOME/skills from a curated list or a GitHub repo path. Use when a user asks to list installable skills, install a curated skill, or install a skill from another repo (including private repos). (file: r1/skill-installer/SKILL.md)
- anthropic-skills:consolidate-memory: Reflective pass over your memory files ? merge duplicates, fix stale facts, prune the index. (file: r2/consolidate-memory/SKILL.md)
- anthropic-skills:doc-coauthoring: Guide users through a structured workflow for co-authoring documentation. Use when user wants to write documentation, proposals, technical specs, decision docs, or similar structured content. This workflow helps users efficiently transfer context, refine content through iteration, and verify the doc works for (file: r2/doc-coauthoring/SKILL.md)
- anthropic-skills:docx: Use this skill whenever the user wants to create, read, edit, or manipulate Word documents (.docx files). Triggers include: any mention of 'Word doc', 'word document', '.docx', or requests to produce professional documents with formatting like tables of contents, headings, page numbers, or letterheads. Also (file: r2/docx/SKILL.md)
- anthropic-skills:new-session-hello: I want claude to read the github repos linked and have some context when I start a new coding session (file: r2/new-session-hello/SKILL.md)
- anthropic-skills:pdf: Use this skill whenever the user wants to do anything with PDF files. This includes reading or extracting text/tables from PDFs, combining or merging multiple PDFs into one, splitting PDFs apart, rotating pages, adding watermarks, creating new PDFs, filling PDF forms, encrypting/decrypting PDFs, extracting (file: r2/pdf/SKILL.md)

- anthropic-skills:pptx: Use this skill any time a .pptx file is involved in any way ? as input, output, or both. This includes: creating slide decks, pitch decks, or presentations; reading, parsing, or extracting text from any .pptx file (even if the extracted content will be used elsewhere, like in an email or summary); editing, (file: r2/pptx/SKILL.md)
- anthropic-skills:schedule: Create or update a scheduled task that runs automatically. Use when the user says things like "every day", "each morning", "remind me in an hour", "run this at noon", or wants to reschedule an existing task. (file: r2/schedule/SKILL.md)
- anthropic-skills:setup-cowork: Guided Cowork setup ? install role-matched plugins, connect your tools, try a skill. (file: r2/setup-cowork/SKILL.md)
- anthropic-skills:skill-creator: Create new skills, modify and improve existing skills, and measure skill performance. Use when users want to create a skill from scratch, edit, or optimize an existing skill, run evals to test a skill, benchmark skill performance with variance analysis, or optimize a skill's description for better triggeri (file: r2/skill-creator/SKILL.md)
- anthropic-skills:xlsx: Use this skill any time a spreadsheet file is the primary input or output. This means any task where the user wants to: open, read, edit, or fix an existing .xlsx, .xlsm, .csv, or .tsv file (e.g., adding columns, computing formulas, formatting, charting, cleaning messy data); create a new spreadsheet from sc (file: r2/xlsx/SKILL.md)
- browser:control-in-app-browser: Control the in-app Browser. Use to open, navigate, inspect, test, click, type, screenshot, or verify local targets such as localhost, 127.0.0.1, ::1, file://, the current in-app browser tab, and websites shown side by side inside Codex. (file: r7/browser/26.623.141536/skills/control-in-app-browser/SKILL.md)
- chrome:control-chrome: Control the user's Chrome browser for tasks that depend on existing Chrome state: tabs, logged-in sessions, or extensions. Prefer purpose-built connectors, APIs, or CLIs when available. (file: r7/chrome/26.623.141536/skills/control-chrome/SKILL.md)
- computer-use:computer-use: Control Windows apps from Codex (file: r7/computer-use/26.623.141536/skills/computer-use/SKILL.md)
- cowork-plugin-management:cowork-plugin-customizer: Customize a Claude Code plugin for a specific organization's tools and workflows. Use when: customize plugin, set up plugin, configure plugin, tailor plugin, adjust plugin settings, customize plugin connectors, customize plugin skill, tweak plugin, modify plugin configuration. (file: r3/cowork-plugin-customizer/SKILL.md)
- cowork-plugin-management:create-cowork-plugin: Guide users through creating a new plugin from scratch in a cowork session. Use when users want to create a plugin, build a plugin, make a new plugin, develop a plugin, scaffold a plugin, start a plugin from scratch, or design a plugin. This skill requires Cowork mode with access to the outputs directory for (file: r3/create-cowork-plugin/SKILL.md)
- design:accessibility-review: Run a WCAG 2.1 AA accessibility audit on a design or page. Trigger with "audit accessibility", "check a11y", "is this accessible?", or when reviewing a design for color contrast, keyboard navigation, touch target size, or screen reader behavior before handoff. (file: r4/accessibility-review/SKILL.md)
- design:design-critique: Get structured design feedback on usability, hierarchy, and consistency. Trigger with "review this design", "critique this mockup", "what do you think of this screen?", or when sharing a Figma link or screenshot for feedback at any stage from exploration to final polish. (file: r4/design-critique/SKILL.md)
- design:design-handoff: Generate developer handoff specs from a design. Use when a design is ready for engineering and needs a spec sheet covering layout, design tokens, component props, interaction states, responsive breakpoints, edge cases, and animation details. (file: r4/design-handoff/SKILL.md)
- design:design-system: Audit, document, or extend your design system. Use when checking for naming inconsistencies or hardcoded values across components, writing documentation for a component's variants, states, and accessibility notes, or designing a new pattern that fits the existing system. (file: r4/design-system/SKILL.md)
- design:research-synthesis: Synthesize user research into themes, insights, and recommendations. Use when you have interview transcripts, survey results, usability test notes, support tickets, or NPS responses that need to be distilled into patterns, user segments, and prioritized next steps. (file: r4/research-synthesis/SKILL.md)
- design:user-research: Plan, conduct, and synthesize user research. Trigger with "user research plan", "interview guide", "usability test", "survey design", "research questions", or when the user needs help with any aspect of understanding their users through research. (file: r4/user-research/SKILL.md)
- design:ux-copy: Write or review UX copy ? microcopy, error messages, empty states, CTAs. Trigger with "write copy for", "what should this button say?", "review this error message", or when naming a CTA, wording a confirmation dialog, filling an empty state, or writing onboarding text. (file: r4/ux-copy/SKILL.md)

- documents:documents: Create, edit, redline, and comment on `.docx`, Word, and Google Docs-targeted document artifacts inside the container, with a strict render-and-verify workflow. Use `render_docx.py` to generate page PNGs (and optional PDF) for visual QA, then iterate until layout is flawless before delivering the final doc (file: r13/documents/26.630.12135/skills/documents/SKILL.md)
- engineering:architecture: Create or evaluate an architecture decision record (ADR). Use when choosing between technologies (e.g., Kafka vs SQS), documenting a design decision with trade-offs and consequences, reviewing a system design proposal, or designing a new component from requirements and constraints. (file: r5/architecture/SKILL.md)
- engineering:code-review: Review code changes for security, performance, and correctness. Trigger with a PR URL or diff, "review this before I merge", "is this code safe?", or when checking a change for N+1 queries, injection risks, missing edge cases, or error handling gaps. (file: r5/code-review/SKILL.md)
- engineering:debug: Structured debugging session ? reproduce, isolate, diagnose, and fix. Trigger with an error message or stack trace, "this works in staging but not prod", "something broke after the deploy", or when behavior diverges from expected and the cause isn't obvious. (file: r5/debug/SKILL.md)
- engineering:deploy-checklist: Pre-deployment verification checklist. Use when about to ship a release, deploying a change with database migrations or feature flags, verifying CI status and approvals before going to production, or documenting rollback triggers ahead of time. (file: r5/deploy-checklist/SKILL.md)
- engineering:documentation: Write and maintain technical documentation. Trigger with "write docs for", "document this", "create a README", "write a runbook", "onboarding guide", or when the user needs help with any form of technical writing ? API docs, architecture docs, or operational runbooks. (file: r5/documentation/SKILL.md)
- engineering:incident-response: Run an incident response workflow ? triage, communicate, and write postmortem. Trigger with "we have an incident", "production is down", an alert that needs severity assessment, a status update mid-incident, or when writing a blameless postmortem after resolution. (file: r5/incident-response/SKILL.md)
- engineering:standup: Generate a standup update from recent activity. Use when preparing for daily standup, summarizing yesterday's commits and PRs and ticket moves, formatting work into yesterday/today/blockers, or structuring a few rough notes into a shareable update. (file: r5/standup/SKILL.md)
- engineering:system-design: Design systems, services, and architectures. Trigger with "design a system for", "how should we architect", "system design for", "what's the right architecture for", or when the user needs help with API design, data modeling, or service boundaries. (file: r5/system-design/SKILL.md)
- engineering:tech-debt: Identify, categorize, and prioritize technical debt. Trigger with "tech debt", "technical debt audit", "what should we refactor", "code health", or when the user asks about code quality, refactoring priorities, or maintenance backlog. (file: r5/tech-debt/SKILL.md)
- engineering:testing-strategy: Design test strategies and test plans. Trigger with "how should we test", "test strategy for", "write tests for", "test plan", "what tests do we need", or when the user needs help with testing approaches, coverage, or test architecture. (file: r5/testing-strategy/SKILL.md)
- github:gh-address-comments: Address actionable GitHub pull request review feedback. Use when the user wants to inspect unresolved review threads, requested changes, or inline review comments on a PR, then implement selected fixes. Use the GitHub app for PR metadata and flat comment reads, and use the bundled GraphQL script via `gh` whe (file: r8/gh-address-comments/SKILL.md)
- github:gh-fix-ci: Use when a user asks to debug or fix failing GitHub PR checks that run in GitHub Actions. Use the GitHub app from this plugin for PR metadata and patch context, and use `gh` for Actions check and log inspection before implementing any approved fix. (file: r8/gh-fix-ci/SKILL.md)
- github:github: Triage and orient GitHub repository, pull request, and issue work through the connected GitHub app. Use when the user asks for general GitHub help, wants PR or issue summaries, or needs repository context before choosing a more specific GitHub workflow. (file: r8/github/SKILL.md)
- github:yeet: Publish local changes to GitHub by confirming scope, committing intentionally, pushing the branch, and opening a draft PR through the GitHub app from this plugin, with `gh` used only as a fallback where connector coverage is insufficient. (file: r8/yeet/SKILL.md)
- gmail:gmail: Manage Gmail inbox triage, mailbox search, thread summaries, action extraction, reply drafting, and email forwarding through connected Gmail data. Use when the user wants to inspect a mailbox or thread, search email with Gmail query syntax, summarize messages, extract decisions and follow-ups, prepare replies (file: r9/gmail/SKILL.md)

- gmail:gmail-inbox-triage: Triage a Gmail inbox into actionable buckets such as urgent, needs reply soon, waiting, and FYI using connected Gmail data. Use when the user asks to triage the inbox, rank what needs attention, find what still needs a reply, or separate important mail from noise. (file: r9/gmail-inbox-triage/SKILL.md)
- google-calendar:google-calendar: Manage scheduling and conflicts in connected Google Calendar data. Use when the user wants to inspect calendars, compare availability, review conflicts, find a meeting room, review event notes or attachments, add or adjust reminders, place temporary holds, or draft exact create, update, reschedule, or canc (file: r10/google-calendar/SKILL.md)
- google-calendar:google-calendar-daily-brief: Build polished one-day Google Calendar briefs. Use when the user asks for today, tomorrow, or a specific date summary with an agenda, conflict flags, free windows, remaining-meeting readouts, or a calendar brief, and the Google Calendar connector is available. (file: r10/google-calendar-daily-brief/SKILL.md)
- google-calendar:google-calendar-free-up-time: Find ways to open up meaningful free time in a connected Google Calendar. Use when the user wants to clear up their day, make room for focus time, create a longer uninterrupted block, or see the smallest set of calendar changes that would give time back. (file: r10/google-calendar-free-up-time/SKILL.md)
- google-calendar:google-calendar-group-scheduler: Find and rank good meeting times for multiple people using connected Google Calendar data. Use when the user wants to schedule a group meeting, compare candidate slots across several attendees, find the best compromise time, or add a room check after narrowing the attendee-compatible options. (file: r10/google-calendar-group-scheduler/SKILL.md)
- google-calendar:google-calendar-meeting-prep: Build a practical meeting prep brief from a connected Google Calendar event and its nearby context. Use when the user wants to prepare for an upcoming meeting, understand what to read beforehand, pull in linked notes or docs, or get a concise brief on what the meeting appears to require. (file: r10/google-calendar-meeting-prep/SKILL.md)
- google-drive:google-docs: Connector-first Google Docs creation and editing in local Codex plugin sessions, with direct native create and batchUpdate workflows for simple docs, DOCX-first import for polished deliverables, target-document checks, smart chip and building-block reconstruction, connector-readback verification, and referenc (file: r11/google-docs/SKILL.md)
- google-drive:google-drive: Use connected Google Drive as the single entrypoint for Drive, Docs, Sheets, and Slides work. Use when the user wants to find, fetch, organize, share, export, copy, or delete Drive files, or summarize and edit Google Docs, Google Sheets, and Google Slides through one unified Google Drive plugin. (file: r11/google-drive/SKILL.md)
- google-drive:google-drive-comments: Write, reply to, and resolve Google Drive comments on Docs, Sheets, Slides, and Drive files with evidence-backed location context. Use when the user asks to leave comments, review a file with comments, respond to comment threads, or resolve Drive comments. (file: r11/google-drive-comments/SKILL.md)
- google-drive:google-sheets: Analyze and edit connected Google Sheets with range precision. Use when the user wants to create Google Sheets, find a spreadsheet, inspect tabs or ranges, search rows, plan formulas, create or repair charts, clean or restructure tables, write concise summaries, or make explicit cell-range updates. (file: r11/google-sheets/SKILL.md)
- google-drive:google-slides: Google Slides work for finding, reading, summarizing, creating, importing, template following, visual cleanup, source-deck adaptation, structural repair, and content edits in native Slides decks. (file: r11/google-slides/SKILL.md)
- pdf:pdf: Read, create, inspect, render, and verify PDF files where visual layout matters. Use Poppler rendering plus Python tools such as reportlab, pdfplumber, and pypdf for generation and extraction. (file: r13/pdf/26.630.12135/skills/pdf/SKILL.md)
- presentations:Presentations: Create or edit PowerPoint or Google Slides decks (file: r13/presentations/26.630.12135/skills/presentations/SKILL.md)
- productivity:memory-management: Two-tier memory system that makes Claude a true workplace collaborator. Decodes shorthand, acronyms, nicknames, and internal language so Claude understands requests like a colleague would. CLAUDE.md for working memory, memory/ directory for the full knowledge base. (file: r6/memory-management/SKILL.md)
- productivity:start: Initialize the productivity system and open the dashboard. Use when setting up the plugin for the first time, bootstrapping working memory from your existing task list, or decoding the shorthand (nicknames, acronyms, project codenames) you use in your todos. (file: r6/start/SKILL.md)
- productivity:task-management: Simple task management using a shared TASKS.md file. Reference this when the user asks about their tasks, wants to add/complete tasks, or needs help tracking commitments. (file: r6/task-management/SKILL.md)
- productivity:update: Sync tasks and refresh memory from your current activity. Use when pulling new assignments from your project tracker into TASKS.md, triaging stale or overdue tasks, filling memory gaps for unknown people or projects, or running a comprehensive scan to catch todos buried in chat and email. (file: r6/update/SKILL.md)

- skill-creator: Create new skills, modify and improve existing skills. Use when users want to create a skill from scratch, edit, or optimize an existing skill. (file: r0/skill-creator/SKILL.md)
- slack:slack: Read Slack context, route to the right Slack workflow, and prepare or perform Slack writes that match the user's intent. (file: r12/slack/SKILL.md)
- slack:slack-channel-summarization: Summarize activity from one Slack channel and return a concise recap, post-ready update, or summary doc. (file: r12/slack-channel-summarization/SKILL.md)
- slack:slack-daily-digest: Create a daily Slack digest from selected channels or topics. Use when the user asks for a daily Slack recap or summary of today's Slack activity. (file: r12/slack-daily-digest/SKILL.md)
- slack:slack-notification-triage: Triage recent Slack activity into a priority queue or task list for the user. (file: r12/slack-notification-triage/SKILL.md)
- slack:slack-outgoing-message: Primary skill for composing, drafting, or refining any outbound Slack content. Use this whenever the task will require using `slack_send_message`, `slack_send_message_draft`, or `slack_create_canvas`. Use `slack` to read or analyze Slack context; use this skill to produce the final outgoing message. (file: r12/slack-outgoing-message/SKILL.md)
- slack:slack-reply-drafting: Draft Slack replies from available context. Use when the user wants help finding messages that likely need a response and preparing reply drafts. (file: r12/slack-reply-drafting/SKILL.md)
- spreadsheets:Spreadsheets: Use this skill when a user requests to create, modify, analyze, visualize, or work with spreadsheet files (`.xlsx`, `.xls`, `.csv`, `.tsv`) or Google Sheets-targeted spreadsheet artifacts with formulas, formatting, charts, tables, and recalculation. (file: r13/spreadsheets/26.630.12135/skills/spreadsheets/SKILL.md)
- template-creator:template-creator: Create or update a reusable personal Codex artifact-template skill. Use when the user invokes $template-creator or asks in natural language to create a template using, from, or based on an attached Word document, PowerPoint presentation, or Excel workbook, or explicitly asks to edit or update a passed arti (file: r13/template-creator/26.630.12135/skills/template-creator/SKILL.md)

How to use skills

- Discovery: The list above is the skills available in this session (name + description + short path). Skill bodies live on disk at the listed paths after expanding the matching alias from `### Skill roots`.
- Trigger rules: If the user names a skill (with `$SkillName` or plain text) OR the task clearly matches a skill's description shown above, you must use that skill for that turn. Multiple mentions mean use them all. Do not carry skills across turns unless re-mentioned.
- Missing/blocked: If a named skill isn't in the list or the path can't be read, say so briefly and continue with the best fallback.
- How to use a skill (progressive disclosure):
  1) After deciding to use a skill, the main agent must expand the listed short `path` with the matching alias from `### Skill roots`, then open and read its `SKILL.md` completely before taking task actions. If a read is truncated or paginated, continue until EOF.
  2) When `SKILL.md` references relative paths (e.g., `scripts/foo.py`), resolve them relative to the directory containing that expanded `SKILL.md` first, and only consider other paths if needed.
  3) If `SKILL.md` points to extra folders such as `references/`, use its routing instructions to identify the files required for the task. The main agent must read each required instruction or reference file itself before acting on it. Do not delegate reading, summarizing, or interpreting skill instructions to a subagent. Subagents may still perform task work when the selected skill allows it.
  4) If `scripts/` exist, prefer running or patching them instead of retyping large code blocks.
  5) If `assets/` or templates exist, reuse them instead of recreating from scratch.
- Coordination and sequencing:
  - If multiple skills apply, choose the minimal set that covers the request and state the order you'll use them.
  - Announce which skill(s) you're using and why (one short line). If you skip an obvious skill, say why.
- Context hygiene:
  - Progressive disclosure applies to selecting relevant files, not partially reading a selected instruction file. Do not load unrelated references, scripts, or assets.
  - Avoid deep reference-chasing: prefer opening only files directly linked from `SKILL.md` unless you're blocked.
  - When variants exist (frameworks, providers, domains), pick only the relevant reference file(s) and note that choice.

- Safety and fallback: If a skill can't be applied cleanly (missing files, unclear instructions), state the issue, pick the next-best approach, and continue.
</skills_instructions>
<plugins_instructions>
Plugins
A plugin is a local bundle of skills, MCP servers, and apps.
How to use plugins
- Skill naming: If a plugin contributes skills, those skill entries are prefixed with `plugin_name:` in the Skills list.
- MCP naming: Plugin-provided MCP tools keep standard MCP identifiers such as `mcp__server__tool`; use tool provenance to tell which plugin they come from.
- Trigger rules: If the user explicitly names a plugin, prefer capabilities associated with that plugin for that turn.
- Relationship to capabilities: Plugins are not invoked directly. Use their underlying skills, MCP tools, and app tools to help solve the task.
- Relevance: Determine what a plugin can help with from explicit user mention or from the plugin-associated skills, MCP tools, and apps exposed elsewhere in this turn.
- Missing/blocked: If the user requests a plugin that does not have relevant callable capabilities for the task, say so briefly and continue with the best fallback.
</plugins_instructions>

2. user (2026-07-08T04:24:01.314Z)

AGENTS.md instructions

<INSTRUCTIONS>
Global instructions (all projects)

Repo-freshness convention: git pull BEFORE reading
Always `git pull` a repo before starting to read it ? at session start for the primary working directory, and again for any sibling/local repo the moment work touches it.

- **Why:** the owner closes every session on a "clean slate", but multiple parallel slates (sessions/machines) exist ? PRs get merged and master moves between sessions, so the local checkout can silently lag the remote truth. Pull first so ramp-up reads (handoff, docs, code) reflect where the remote actually is.
- **How (safe form):** `git pull --ff-only` on the current branch (a plain fetch+ff). Never auto-merge, rebase, or discard local state to make a pull succeed.
- **If it can't fast-forward** (diverged branch, dirty tree in the way): do NOT force anything ? report the state to the owner and proceed read-only on what's local, flagging that it may be stale. Uncommitted changes are often a sibling session's or the owner's WIP; never clobber them.

Git branching safety: concurrent sessions / shared checkouts
Multiple sessions ? and spawned/background tasks ? may share ONE working checkout and create branches + commit **concurrently**, so `git branch` / `git log` can change UNDER you between two commands. (Spawned "fresh worktree" tasks are NOT always isolated.) This has caused a real incident: a sibling commit landed in the gap between a branch-point check and `git checkout -b`, so the new branch rode on top of it and a **squash-merge silently absorbed the sibling's work**. Protocol ? do this EVERY time, not only when you suspect a collision:
- **Branch from the REMOTE, explicitly:** `git fetch origin` then `git checkout -b <name> origin/<base>`
  (e.g. `origin/master`). Never assume the current branch is the intended base.
- **Re-verify right before `git add`/commit:** `git branch --show-current` + `git log --oneline -1`
  (HEAD is yours). Before opening a PR, `git log --oneline origin/<base>..HEAD` must list ONLY your
  commits ? if a sibling commit appears, re-cut the branch from `origin/<base>`.
- **Stage explicit paths** (`git add <files>`) ? never `git add -A`/`.`/`-u`, so sibling or untracked
  files can't ride into your commit.
- **Serialize repo writes:** don't start a repo-writing spawned/background task while you hold

  uncommitted work ? commit or push first. If one may be running, treat the tree + current branch
  as able to shift, and put this branching-safety reminder into the spawned task's own prompt.

Session-handoff convention

Maintain a living **session handoff** for each project and use it to resume context quickly across windows.

- **Where it lives:**
  - If the project has an auto-memory dir (`~/.Codex/projects/<project>/memory/`), keep the handoff as `session-handoff.md` there, and list it under a **"? Start here"** entry at the top of that project's `MEMORY.md`.
  - Otherwise, keep a `SESSION_HANDOFF.md` at the repo root.
- **What it contains** (keep it to ~1 page ? it POINTS to detailed artifacts, never duplicates them):
  1. **You are here** ? current state.
  2. **Next steps / open threads.**
  3. **Ramp-up kit** ? the exact files/docs to read to get current.
  4. **Key decisions** ? so they aren't relitigated.
- **At session start:** if a handoff exists, read it before starting substantive work, and use it
  (plus its ramp-up kit) to get up to speed instead of replaying past conversation.
- **Updating it:** keep it current **automatically** ? refresh at meaningful checkpoints (a PR is
  pushed/merged, a key decision is made, work shifts phase, or a session is wrapping up), so a restart can ramp up without losing context. Do **not** rewrite it every turn ? only when something worth resuming from changed. The phrases **"update the handoff"** / **"refresh the handoff"** force an immediate full rewrite on demand.
- **Keep it honest:** it reflects real, shipped state ? never aspirational. (See each project's attribution/working-agreement note where present.)

Code review ? pre-LGTM gate (applies to every PR/code review, every project)

Do **not** post **LGTM** or otherwise sign off on a pull request until I have personally **verified** the following ? never trust the PR description's claims, reproduce them locally:

1. **Full test suite is green.** Run the whole suite (e.g. `run_all_tests.py` / `pytest`), not just the changed files.
2. **Install any dependency needed** so the suite actually collects and runs ? do not let modules skip or error for missing deps. (If a check genuinely can't run locally ? GPU/WSL/real-video/etc. ? say so explicitly and note what was and wasn't verified.)
3. **Lint + type checks are clean** ? `ruff check .` and `mypy ...`, matching the repo's CI. Install `ruff`/`mypy` if absent.
4. **Coverage stays up** ? run with coverage and confirm it stays **at or above the repo's floor** (e.g. the CI `--cov-fail-under` value) and does **not regress** versus the base branch.
5. **Review findings are resolved** ? bugs and repo-convention/discipline issues addressed.

LGTM is a *verified* stamp, never a rubber stamp. State plainly what was and wasn't verifiable locally.
</INSTRUCTIONS>
<environment_context>
  <cwd>C:\Users\avidu\Projects\Agent Sessions</cwd>
  <shell>powershell</shell>
  <current_date>2026-07-08</current_date>
  <timezone>Asia/Calcutta</timezone>
  <filesystem><workspace_roots><root>C:\Users\avidu\Projects\Agent Sessions</root></workspace_roots><permission_profile type="disabled"><file_system type="unrestricted" /></permission_profile></filesystem>
</environment_context>

3. user (2026-07-08T04:24:01.368Z)

Can you tell me what are all these "Agent Session ..." directories in the C:\Users\avidu\Projects folder? can we clean them up and make sure we do all this inside the

root folder Projects/Agent Sessions

4. assistant (2026-07-08T04:24:08.654Z)

I?ll first check whether the current `Projects/Agent Sessions` folder is a git repo and fast-forward it if so, then I?ll inventory the sibling `Agent Session...` directories before suggesting any cleanup. I?ll avoid deleting or moving anything until we?ve identified what?s safe.

5. assistant (2026-07-08T04:24:23.821Z)

The repo is indeed `C:/Users/avidu/Projects/Agent Sessions`, but `git pull --ff-only` stopped because local `SESSION_HANDOFF.md` changes would be overwritten. Per your repo convention I?ll treat the local checkout as possibly stale, read the handoff, and keep the cleanup discovery non-destructive.

6. assistant (2026-07-08T04:24:44.235Z)

I found ten matching folders, including the intended root. The root is 77 commits behind `origin/main` and has an uncommitted handoff edit, so I?m going to identify whether the sibling folders are git worktrees/repo checkouts and whether any contain uncommitted work before proposing the cleanup.

7. assistant (2026-07-08T04:24:57.859Z)

These folders are all registered git worktrees of the same `agent-sessions` repo, not random directories. I?m going to run the safe `git pull --ff-only` check in each sibling worktree now, then compare each branch to `origin/main` so we can separate ?already merged / disposable? from ?still has unique work.?

8. assistant (2026-07-08T04:25:34.158Z)

The sibling worktrees themselves are clean: `git status` only showed branch tracking lines, no modified or untracked files. Most of their upstream branches are already gone, which usually means the PR branch was deleted after merge; I?m checking the commit graph now rather than guessing from names.

9. assistant (2026-07-08T04:26:00.266Z)

The table display was too narrow to be useful, so I?m re-running the graph comparison as structured output. The key fields are `WorktreeOnlyCommits` and `MergedToOriginMain`; zero worktree-only commits means the folder can be removed as a worktree without losing branch-only commits.

10. assistant (2026-07-08T04:27:02.409Z)

One more wrinkle: the main root has a local `SESSION_HANDOFF.md` edit, so I can?t fast-forward it without deciding what to do with that edit. The sibling directories are independent and clean, though, so the cleanup path is straightforward: unregister/remove those worktrees and keep `C:/Users/avidu/Projects/Agent Sessions` as the only `Projects` checkout.

11. assistant (2026-07-08T04:27:14.222Z)

I?ve verified the exact absolute paths and that every sibling worktree has zero commits not already in `origin/main`. I?m going to remove just those registered worktree folders now; I?m leaving the main root and its local handoff edit alone.

12. assistant (2026-07-08T04:27:38.811Z)

`git worktree remove` unregistered the sibling worktrees, but Windows refused to delete the physical folders and their stale `.git/worktrees` metadata with ?Permission denied.? I?m going to do a tighter cleanup pass using exact resolved paths and PowerShell?s native deletion, with guardrails so only those `C:\Users\avidu\Projects\Agent Sessions - ...` directories are touched.

13. assistant (2026-07-08T04:28:26.524Z)

Cleanup succeeded on the `Projects` side: only `C:\Users\avidu\Projects\Agent Sessions` remains there now. I?m doing one final status check so I can call out the remaining root state accurately.

14. assistant (2026-07-08T04:28:41.559Z)

Those `C:\Users\avidu\Projects\Agent Sessions - ...` folders were old git worktrees for prior `agent-sessions` branches, not separate projects. I checked each one: they were clean, already merged into `origin/main`, and had zero branch-only commits.

I removed all 9 sibling worktree folders and stale git worktree metadata. Under `C:\Users\avidu\Projects`, only this remains now:

`C:\Users\avidu\Projects\Agent Sessions`

One remaining note: the root repo is still `77` commits behind `origin/main` because `git pull --ff-only` is blocked by a local edit to [SESSION_HANDOFF.md](<C:/Users/avidu/Projects/Agent Sessions/SESSION_HANDOFF.md>). I left that untouched. There are also still registered worktrees under `C:\Users\avidu\Documents\Codex\...`; I did not touch those since your cleanup request was specifically for `Projects`.

15. user (2026-07-08T04:29:36.960Z)

please sync to remote head

16. assistant (2026-07-08T04:29:48.851Z)

I?ll sync the root checkout by preserving the local handoff edit in a stash first, then fast-forwarding `main` to `origin/main`. That keeps the repo at remote head without throwing away the uncommitted note.

17. assistant (2026-07-08T04:30:10.952Z)

Synced [C:/Users/avidu/Projects/Agent Sessions](<C:/Users/avidu/Projects/Agent Sessions>) to remote head.

`main` fast-forwarded from `5d81234` to `80b91d5`, and the working tree is clean:

```text
## main...origin/main
```

I preserved the previous local `SESSION_HANDOFF.md` edit in a stash:

```text
stash@{0}: On main: pre-sync local handoff edit 2026-07-08T09:59:57.9548571+05:30
```

18. user (2026-07-08T04:30:41.217Z)

can you do a audit of the work and report and tech debts, follow ups and doc updates which will improve

19. assistant (2026-07-08T04:30:50.103Z)

I?ll use the engineering tech-debt skill here, since you?re asking for an audit with debt, follow-ups, and documentation improvements. I?ll first re-sync safely per the repo rule, then read the handoff and current docs/code shape before running targeted checks.

20. assistant (2026-07-08T04:30:58.758Z)

The repo is already at remote head. I?m reading the handoff and the tech-debt workflow now so the audit is grounded in the project?s current state rather than just scanning filenames.

21. assistant (2026-07-08T04:31:12.194Z)

The handoff says the knowledge/replay tracker is complete and most remaining work is empirical or polish. I?m going to inspect the repo?s own instructions, docs, test plan, and CLI/code hotspots next, then I?ll run the local gates that are practical for an audit.

22. assistant (2026-07-08T04:31:26.595Z)

The archived tech-debt plan is fully marked done, so this audit needs to look for the next layer: debt introduced or exposed by the K0-K12 knowledge/replay work, docs that lag current behavior, and gates that are still empirical rather than automated.

23. assistant (2026-07-08T04:31:33.877Z)

The repo has grown into a medium-sized Python CLI now: about two dozen modules and nearly thirty test files. I?m starting the verification gates, then I?ll combine that evidence with manual review of the replay/handoff/doc surfaces.

24. assistant (2026-07-08T04:32:27.169Z)

The full suite is green: 576 passed with 96.94% coverage, comfortably above the 80% floor. I?m running lint and typing now, then I?ll inspect the command outputs and docs for inconsistencies or places where the docs could carry the next phase better.

25. assistant (2026-07-08T04:32:56.100Z)

Lint and type checks are clean. The broad text scan hit the archive volume and timed out, so I?m narrowing it to source, docs, tests, config, and scripts to keep the signal useful.

26. assistant (2026-07-08T04:33:05.764Z)

The CLI help matches the current feature set, which is good. I?m going to run the repo?s own baseline health commands next, because they?ll surface the project-specific warnings that ordinary pytest/ruff/mypy won?t.

27. assistant (2026-07-08T04:35:02.798Z)

The project-specific gates add two useful signals: baseline eval is healthy but replay value is still gated, and archive status says the local archive is behind the visible stores by 1,102 new files plus 1 changed file. I?m checking the handoff/replay command outputs and the relevant docs around those boundaries now.

28. assistant (2026-07-08T04:35:15.712Z)

A doc inconsistency is already emerging: the handoff says the knowledge/replay tracker is complete/current, while the README and docs index still label the older baseline loop closure as active. I?m checking the core project docs and generated baseline files so the recommendations can be precise instead of vague ?update docs? handwaving.

29. assistant (2026-07-08T04:35:26.736Z)

The older loop-closure doc is genuinely stale: its tracker still has undone rows that have been implemented or superseded, and it says the handoff points there even though the handoff now points to the knowledge/replay plan. I?m checking the generated health artifacts next to make sure the code?s reality agrees with the docs.

30. assistant (2026-07-08T04:36:14.215Z)

No files were changed by the audit commands. The dry-run replay selection is interesting: it selects 20 replay candidates cleanly, but the top set is clustered around old Claude sessions

with hashed `wf_...` project slugs, which points to a next-layer quality improvement rather than a correctness bug.

31. assistant (2026-07-08T04:37:03.981Z)

I?ve got enough local evidence for the code/docs audit. To avoid treating stale issue references as truth, I?m doing one quick GitHub issue/PR check now, then I?ll summarize priorities with verification results and suggested doc updates.

32. assistant (2026-07-08T04:37:13.739Z)

GitHub confirms there are no open PRs. There are open issues for the completed knowledge/replay umbrellas (#23/#25/#26), plus explicit follow-up issues #59/#60 and archive backfill #32, so part of the audit is issue hygiene as well as code.

33. assistant (2026-07-08T04:38:05.155Z)

I audited the repo at `main` / `80b91d5`. No files changed.

Verification
- Full tests: `576 passed`, coverage `96.94%`.
- `ruff check .`: clean.
- `mypy agent_sessions tools tests`: clean.
- `baseline lint --dry-run`: `0` errors, `2` warnings.
- `baseline eval --dry-run`: `14` pass, `0` fail, `2` gated.
- Open PRs: none.
- Git working tree: clean.

Highest-Value Debt
Priority	Item	Why It Matters	Effort
P0	Archive is stale locally: `1,102` new visible files, `1` changed visible file.	Baseline/replay quality depends on archive freshness; current mining misses recent sessions.	M
P0	Docs point at stale ?active? work. [README.md](<C:/Users/avidu/Projects/Agent Sessions/README.md:183>), [docs/README.md](<C:/Users/avidu/Projects/Agent Sessions/docs/README.md:8>), and [BASELINE_LOOP_CLOSURE.md](<C:/Users/avidu/Projects/Agent Sessions/docs/BASELINE_LOOP_CLOSURE.md:3>) lag the shipped K0-K12 state.	New sessions will ramp up from wrong priorities.	S
P1	Redaction is still v0 denylist-only. See [baseline_redaction.py](<C:/Users/avidu/Projects/Agent Sessions/agent_sessions/baseline_redaction.py:38>) and issue [#59](https://github.com/avidullu/agent-sessions/issues/59).	Replay bundles are the one path that can leave the machine; blind spots include JWTs, Stripe/Google keys, high-entropy blobs.	M
P1	Replay selection quality is deterministic but shallow. [baseline_replay.py](<C:/Users/avidu/Projects/Agent Sessions/agent_sessions/baseline_replay.py:101>) uses simple basename slugs, and selection sorts by score/path at [line 287](<C:/Users/avidu/Projects/Agent Sessions/agent_sessions/baseline_replay.py:287>).	Current top 20 are clustered old Claude sessions with opaque `wf_...` slugs, so first replay run may be low-value.	M
P1	Issue hygiene: #23, #25, #26 are still open even though K0-K12 landed; #59/#60/#32/#19 remain genuine follow-ups.	Keeps planning state honest across GitHub, docs, and handoff.	S
P2	`baseline handoffs audit` report text still says K7 ?will own? proposal generation after K7 shipped. [baseline_handoffs.py](<C:/Users/avidu/Projects/Agent Sessions/agent_sessions/baseline_handoffs.py:769>).	Small but confusing generated-report drift.	S
P2	Automation scripts push by default and assume the current checkout is safe. [daily-export.ps1](<C:/Users/avidu/Projects/Agent Sessions/scripts/daily-export.ps1:13>), [daily-export.sh](<C:/Users/avidu/Projects/Agent Sessions/scripts/daily-export.sh:37>).	Fine for trusted local jobs, but add branch/dirty-tree/lock checks before relying on them daily.	S-M

Recommended Follow-Ups
1. Run an intentional archive sync PR or approved direct sync: `export --all --pdf`, inspect

`archive/` changes, commit explicit paths.
2. Do a docs-only cleanup PR: update README/docs index, mark `BASELINE_LOOP_CLOSURE` done or turn it into the backlog umbrella, and align §7 statuses.
3. Close or comment on GitHub issues #23/#25/#26 as completed by PRs #45-#57; keep #59, #60, #32, #19 open.
4. Implement issue #60 in small pieces: richer replay slugs, clean bundle reruns, project-scoped replay proposals, stronger R2/H1 gates.
5. Implement issue #59 before handing replay bundles to another agent for real.
6. Decide what to do with the two known lint orphan warnings: add generated blocks/links for `agent-sessions` and `avidullu`, or document them as intentional.

Doc Updates
- [README.md](<C:/Users/avidu/Projects/Agent Sessions/README.md:190>) should mention the newer commands: `baseline lint`, `baseline eval`, `baseline handoffs`, and `baseline replay`.
- [BASELINE_LOOP_CLOSURE.md](<C:/Users/avidu/Projects/Agent Sessions/docs/BASELINE_LOOP_CLOSURE.md:121>) needs its definition of done checked off where reality now passes.
- [docs/README.md](<C:/Users/avidu/Projects/Agent Sessions/docs/README.md:8>) should stop labeling the old loop closure as the active project.
- `baseline handoffs audit` generated copy should be refreshed from future-tense K6/K7 language to current ownership.

34. user (2026-07-08T04:38:05.634Z)

please make a PR of your changes and send for review

35. assistant (2026-07-08T04:38:19.010Z)

I didn?t leave any file edits from the audit itself, so I?ll turn the highest-confidence audit cleanup into a small docs/report PR: capture the audit findings, fix the stale ?active project? pointers, and update the generated handoff-audit wording. Then I?ll push a branch and open a draft PR for review.

36. assistant (2026-07-08T04:38:24.687Z)

I?m using the GitHub publish workflow skill for this PR step, so I?ll follow the repo?s branch-from-remote and explicit-staging rules carefully.

37. assistant (2026-07-08T04:38:39.099Z)

Before editing, I?m creating a fresh branch from `origin/main` rather than from the current checkout. The scope will stay small: docs/report cleanup plus one generated-report wording fix with its test expectation.

38. assistant (2026-07-08T04:38:51.563Z)

I?m going to patch the docs first, then adjust the handoff audit renderer and its tests so the committed report no longer talks as if K6/K7 are future work.

39. assistant (2026-07-08T04:39:14.211Z)

I found no existing test assertion around the stale K7 wording, so I?m adding one while keeping the code change tiny. The audit report itself will become a tracked docs artifact rather than only living in chat.

40. assistant (2026-07-08T04:40:29.765Z)

The first patch is in place: one audit report, docs pointer cleanup, and a small report-rendering wording fix. I?m reviewing the diff now before running checks.

41. assistant (2026-07-08T04:40:42.061Z)

The diff looks scoped. I?m running the repo gates now, including the full suite because this

touches a Python renderer and a test.

42. assistant (2026-07-08T04:41:13.374Z)

The main gates passed. I?m running `baseline eval` as the final project-specific check, then I?ll re-verify branch/HEAD before staging exactly the touched files.

43. assistant (2026-07-08T04:41:39.829Z)

GitHub CLI is available in this repo workflow, and I?ll use it for the publish step unless the connector path is cleaner. Before staging, I?m checking auth and re-verifying that the branch contains only this work.

44. assistant (2026-07-08T04:41:44.318Z)

The branch is still based directly on `origin/main` and all uncommitted files are in scope. I?m staging only those explicit paths now.

45. assistant (2026-07-08T04:41:49.944Z)

Staged paths are exactly the audit/docs/reporting files. I?m committing that now, then I?ll push and open the PR.

46. assistant (2026-07-08T04:41:54.836Z)

The commit is made. I?m doing the required pre-PR branch check now: current branch, HEAD, and `origin/main..HEAD` should show only this commit.

47. assistant (2026-07-08T04:41:59.164Z)

Only my audit-doc commit is ahead of `origin/main`, so I?m pushing the branch now.

48. assistant (2026-07-08T04:42:06.806Z)

Branch is pushed. I?m checking for the GitHub PR tool first; if it isn?t exposed cleanly here, I?ll use `gh pr create` with an explicit body file.

49. assistant (2026-07-08T04:42:12.568Z)

I found the GitHub PR tool, so I?ll open the PR through the connector now as a ready-for-review PR, not a draft, because you asked to send it for review.

50. assistant (2026-07-08T04:42:24.027Z)

The connector returned a GitHub 404 for PR creation even though the pushed branch exists, so I?m falling back to `gh pr create` with the same title/body.

51. assistant (2026-07-08T04:42:44.825Z)

PR #61 is open. One small thing remains from the repo?s handoff convention: I?m adding a stable handoff note that the audit report now exists and where the durable follow-ups live, without baking in ephemeral ?PR is open? text.

52. assistant (2026-07-08T04:43:16.026Z)

I added the stable handoff note. Since that is docs-only, I?ll review it, commit it separately, push it to the already-open PR, and update the PR body so reviewers see it.

53. assistant (2026-07-08T04:43:20.850Z)

The handoff diff is only durable context, no code. I?m staging just `SESSION_HANDOFF.md` and committing it as the second PR commit.

54. assistant (2026-07-08T04:43:26.031Z)

Second commit is in. I?m checking the branch range again before pushing, then I?ll update the PR body to mention the handoff refresh.

55. assistant (2026-07-08T04:43:34.662Z)

The updated branch is pushed. I?m editing PR #61?s body so the review summary matches the final two-commit diff.

56. assistant (2026-07-08T04:43:51.742Z)

PR body is updated. I?m doing one final clean-state and PR snapshot check, then I?ll hand you the link and the validation summary.

57. assistant (2026-07-08T04:44:04.928Z)

Opened PR #61 for review: [docs: record audit follow-ups](https://github.com/avidullu/agent-sessions/pull/61)

It?s ready for review, not a draft. Branch: `codex/audit-doc-cleanup`.

Included:
- New audit report: `docs/WORK_AUDIT_2026-07-08.md`
- README/docs index cleanup for current baseline status
- `BASELINE_LOOP_CLOSURE.md` marked historical/done with backlog split out
- Handoff audit wording refreshed plus a regression assertion
- `SESSION_HANDOFF.md` updated with durable audit/follow-up pointers

Validation passed:
- `pytest --cov`: 576 passed, 96.94%
- `ruff`: clean
- `mypy`: clean
- `baseline lint`: 0 errors, 2 known warnings
- `baseline eval`: 14 pass, 0 fail, 2 gated

::git-create-branch{cwd="C:\Users\avidu\Projects\Agent Sessions" branch="codex/audit-doc-cleanup"}
::git-stage{cwd="C:\Users\avidu\Projects\Agent Sessions"}
::git-commit{cwd="C:\Users\avidu\Projects\Agent Sessions"}
::git-push{cwd="C:\Users\avidu\Projects\Agent Sessions" branch="codex/audit-doc-cleanup"}
::git-create-pr{cwd="C:\Users\avidu\Projects\Agent Sessions" branch="codex/audit-doc-cleanup" url="https://github.com/avidullu/agent-sessions/pull/61" isDraft=false}